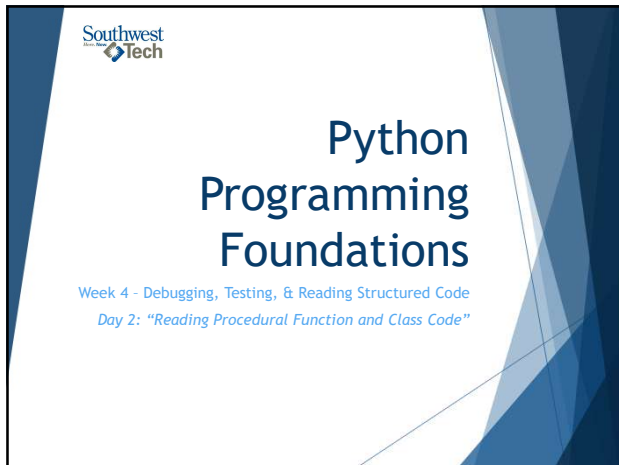
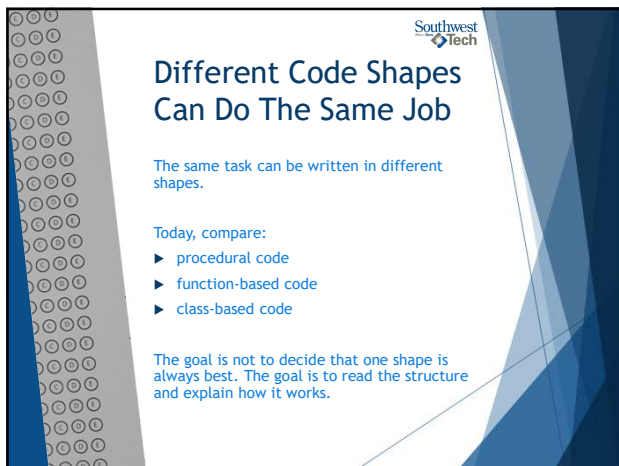
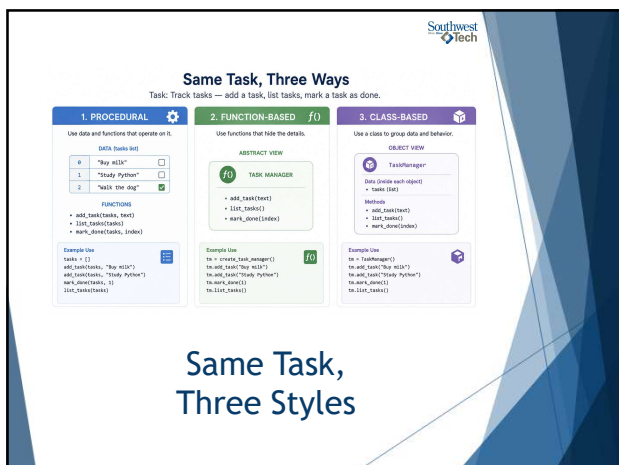




1



2



3



Build Structure, Then Read Structure

Week 3 focused on building structure.

Week 4 adds code literacy: reading structure someone else wrote.

Code reading matters because:

- ▶ examples may use unfamiliar organization
- ▶ AI may generate class-based code
- ▶ documentation may assume structure vocabulary
- ▶ debugging depends on knowing where to look

4



What Counts as Success Today

Read the structure ->
identify the parts ->
explain in plain language ->
modify one small thing.

For structured code, ask:

- What stores data?
- What performs actions?
- Where does the program start?
- What line changes behavior?

5



A Simple Reading Path

Understand first. Then change one small thing with confidence.

1 READ THE STRUCTURE

Get the big picture. What does this do?

```
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print(average)
```

What is the overall flow from start to finish?

2 IDENTIFY THE PARTS

Name the important pieces. What is each part for?

```
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print(average)
```

Can you label each part with its job?

3 EXPLAIN IN PLAIN LANGUAGE

Say what it does out loud. Use your teaching to.

- Start with a total of 0.
- Look at each score in the list.
- Add the score to the running total.
- Find the average by dividing the total by how many scores there are.
- Print the average.

Could someone else understand your explanation?

4 MODIFY ONE SMALL THING

Make one tiny change. Then run it and see.

Example small change: Round the average to 2 decimals.

```
average = total / len(scores)
average = round(average, 2)
```

Did it work? What happened?

The Goal: Read with purpose. Understand deeply. Change with confidence.

Remember: Small steps lead to strong understanding.

Today's Success

6

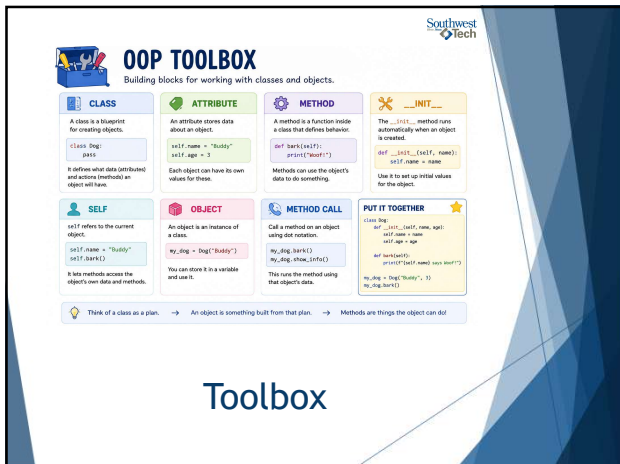


What We Will Use Today

- ▶ procedural code
- ▶ functions
- ▶ classes
- ▶ attributes
- ▶ methods
- ▶ `__init__`
- ▶ `self`
- ▶ object creation

These are recognition tools for reading small structured examples.

7



OOP TOOLBOX

Building blocks for working with classes and objects.

CLASS A class is a blueprint for creating objects. <pre>class Dog: pass</pre> It defines what data (attributes) and actions (methods) an object will have.	ATTRIBUTE An attribute stores data about an object. <pre>self.name = "Buddy" self.age = 3</pre> Each object can have its own values for these.	METHOD A method is a function inside a class that defines behavior. <pre>def bark(self): print("bark!")</pre> Methods can use the object's data to do something.	__INIT__ The <code>__init__</code> method runs automatically when an object is created. <pre>def __init__(self, name): self.name = name</pre> Use it to set up initial values for the object.
SELF <code>self</code> refers to the current object. <pre>self.name = "Buddy" self.bark()</pre> It lets methods access the object's own data and methods.	OBJECT An object is an instance of a class. <pre>my_dog = Dog("Buddy")</pre> You can store it in a variable and use it.	METHOD CALL Call a method on an object using dot notation. <pre>my_dog.bark() my_dog.show_info()</pre> This runs the method using that object's data.	PUT IT TOGETHER <pre>class Dog: def __init__(self, name, age): self.name = name self.age = age def bark(self): print(f"{self.name} says bark!") my_dog = Dog("Buddy", 3) my_dog.bark()</pre>

Think of a class as a plan. → An object is something built from that plan. → Methods are things the object can do

Toolbox

8



What We Will "Skip" For Now

- ▶ Inheritance
- ▶ Polymorphism
- ▶ Decorators
- ▶ large class hierarchies

Today's goal is smaller:

- ▶ identify the class
- ▶ identify what it stores
- ▶ identify what it does
- ▶ make one small change

9



PARKED FOR LATER

• Useful later, not today's focus. •

INHERITANCE



One class can inherit attributes and methods from another class.

POLYMORPHISM



Different objects can respond to the same method call in their own way.

DECORATORS



A way to add extra behavior to functions or methods.

LARGE CLASS HIERARCHIES



Designing and managing many related classes in a bigger structure.

We'll use these ideas soon.
For now, let's build a strong foundation.

Parked For Later

10

Procedural Code: Steps In Order

Procedural code is organized as steps in order.

It often reads like:

- ▶ create values
- ▶ change values
- ▶ loop or branch
- ▶ print output

This can be clear for small programs, but it may become harder to manage as the program grows.



11



PROCEDURAL STEPS

One task. Four clear steps.

1 CREATE LIST



Start with an empty list to hold tasks.

2 ADD TASKS



Add tasks to the list one at a time.

3 LOOP OVER TASKS



Go through each task in the list, one by one.

4 PRINT OUTPUT



Show each task as output.

✓ Goal: Store tasks, go through them, and display them.

Procedural Coding

12



13



14



15

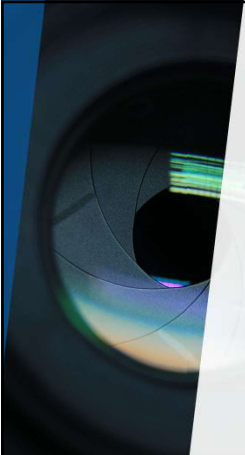


What to Watch For

The “ordered” steps”

- ▶ where the list is created
- ▶ where tasks are added
- ▶ where tasks are displayed
- ▶ what would be repeated if the program grew

16



Demo #2

“Function Based Task Tracker”

17



What to Watch For

Function-based code names responsibilities.

In the demo, identify:

- ▶ the function that adds a task
- ▶ the function that shows tasks
- ▶ where each function is called
- ▶ what data is passed in

18

Discount Calculation
Item price: \$100
Discount: 20%

Expected
\$80.00
 $100 - 20\% \text{ of } 100 = 80$

Actual
\$120.00
This doesn't look right.

Let's find where the calculation went wrong.

Calculation Steps

Step	Description	Value
1	Start with item price	100.00
2	Calculate discount (20% of 100)	20.00
3	Apply discount to price $\text{price} = \text{price} + \text{discount}$	120.00
4	Tax (0%)	0.00
5	Final price	120.00

The discount should be subtracted, not added. $\text{price} = \text{price} - \text{discount}$

**Demo 2:
Function Based Task Tracker**

19

Class Based Coding

This Photo by Unknown Author is licensed under CC BY 4.0

20

A Class Stores Data And Actions Together

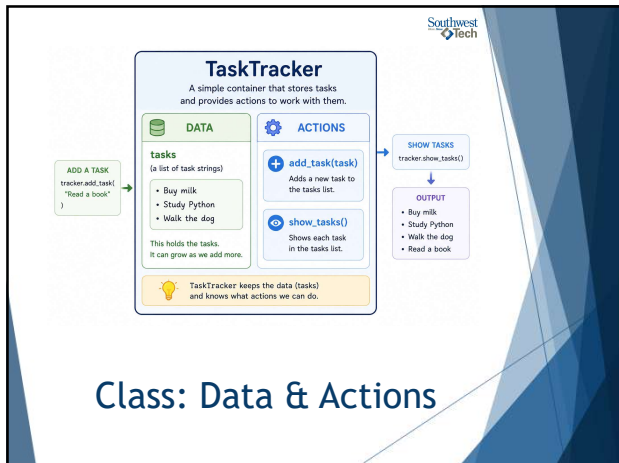
A class can group data and actions that belong together.

Beginning terms:

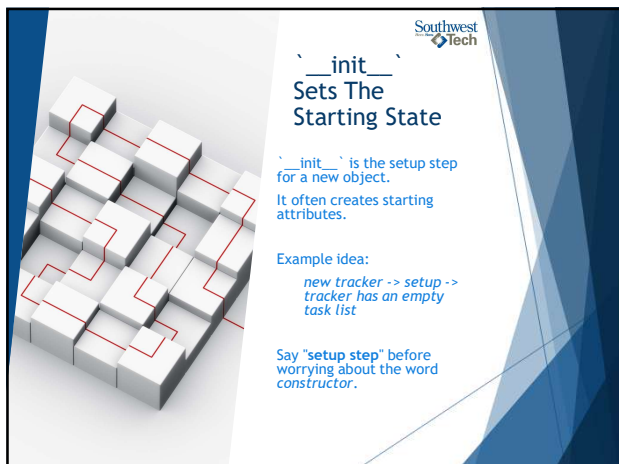
- ▶ **attribute**: data the object stores
- ▶ **method**: action the object can do
- ▶ **object**: one usable instance of the class

This is organization, not magic.

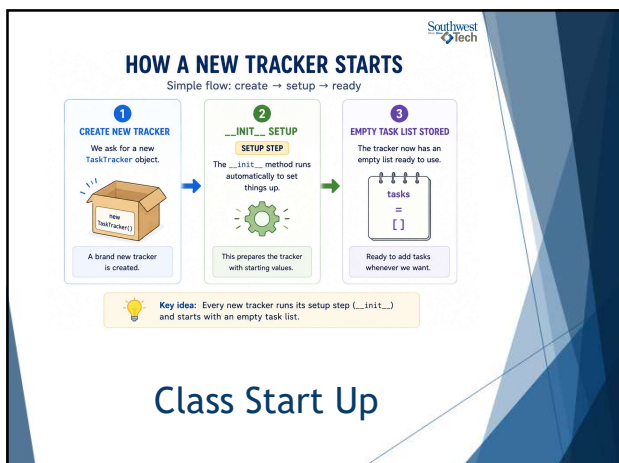
21



22



23



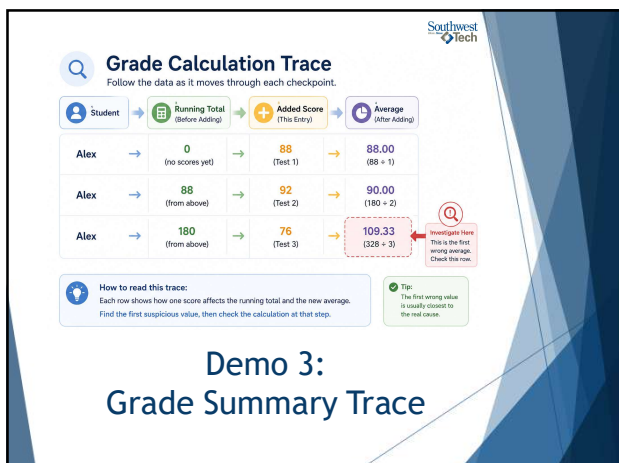
24



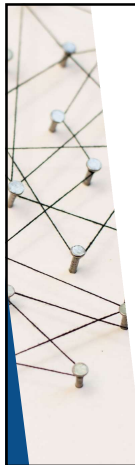
25



26



27



What Does `self` Mean?

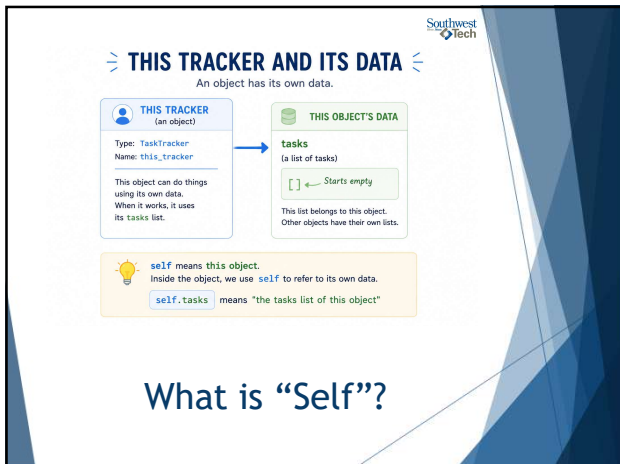
At an introductory level, `self` means "this object."

`self.tasks` means this object has a `tasks` attribute.

When a method runs, it uses `self` to access the object's stored data.

Do not overthink it today. Recognize the pattern.

28



THIS TRACKER AND ITS DATA

An object has its own data.

THIS TRACKER (an object)

Type: TaskTracker
Name: this_tracker

This object can do things using its own data. When it works, it uses its tasks list.

THIS OBJECT'S DATA

tasks (a list of tasks)

[] ← Starts empty

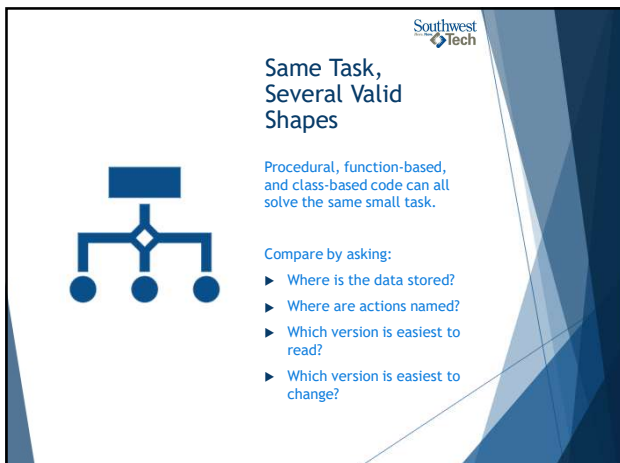
This list belongs to this object. Other objects have their own lists.

self means this object. Inside the object, we use **self** to refer to its own data.

self.tasks means "the tasks list of this object"

What is "Self"?

29



Same Task, Several Valid Shapes

Procedural, function-based, and class-based code can all solve the same small task.

Compare by asking:

- ▶ Where is the data stored?
- ▶ Where are actions named?
- ▶ Which version is easiest to read?
- ▶ Which version is easiest to change?

30

Southwest
Tech

Common Failure: Prestige Bias

Class-based code is not automatically better.

Choose or evaluate structure by fit:

- ▶ Does the structure match the problem?
- ▶ Can you explain it?
- ▶ Can you modify it safely?
- ▶ Is it small enough for this course target?

Advanced-looking code is not useful if you cannot understand it.

31

Southwest
Tech

CHOOSE WHAT FITS THE PROBLEM

Good design is about clarity and flexibility, not complexity.

ADVANCED-LOOKING
Uses more structure than needed for this simple task.

```
class TaskManager:
    def __init__(self):
        self.tasks = []
        self.listeners = []
    def add_task(self, task):
        self._validate_task(task)
        self._add_listener(task)
        self._log_change()
    def show_tasks(self):
        for t in self.tasks:
            print(f"{t}")
# ... more helper methods ...
```

More parts to learn
More to maintain and test
Harder to explain at a glance

THE GOAL
Track a small list of tasks in memory and show them.



CHOOSE FIT-FOR-PURPOSE
Because it is:

- ✔ Explorable
Easy to read and understand.
- ✔ Modifiable
Easy to change and extend.

FITS THE PROBLEM
Simple, direct, and easy to work with for what we need right now.

```
tasks = []
def add_task(task):
    tasks.append(task)
def show_tasks():
    for t in tasks:
        print(f"{t}")
```

Clear and easy to explain
Anyone can see what it does.
Simple to modify
Change or add features quickly.
Just enough structure
Matches the size of the problem.

GOOD RULE: Start simple. Add structure only when the problem grows.
The best code is the code that is easy to understand and easy to change.

Common Failure: Prestige Bias

32

Southwest
Tech

Common AI Failure!

In AI Generated code:

The AI may return class-based solutions that *look polished* but exceed the current need.

33



Assignment #7

For Assignment 7, read and lightly modify a structured example.

Your work should show that you can:

- ▶ identify the class
- ▶ identify attributes
- ▶ identify methods
- ▶ create or use an object
- ▶ call a method
- ▶ make one small working change

34



Structured Code Reading Checklist



Before editing class-based code, answer:

- ▶ What is the class name?
- ▶ What does the object store?
- ▶ What actions can it do?
- ▶ Where is the object created?
- ▶ Where are methods called?
- ▶ What one change am I making?

35



Structure Code Checklist

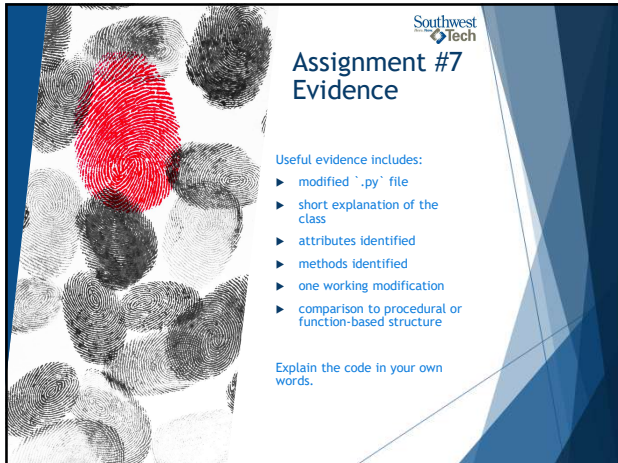
MY CLASS CHECKLIST

Did I build it, use it, and make a change?

- ☒ 1 class name
- ☒ 2 stored data
- ☒ 3 actions
- ☒ 4 object creation
- ☒ 5 method calls
- ☒ 6 one change

 Small steps. Clear code. Real learning.

36



Southwest Tech

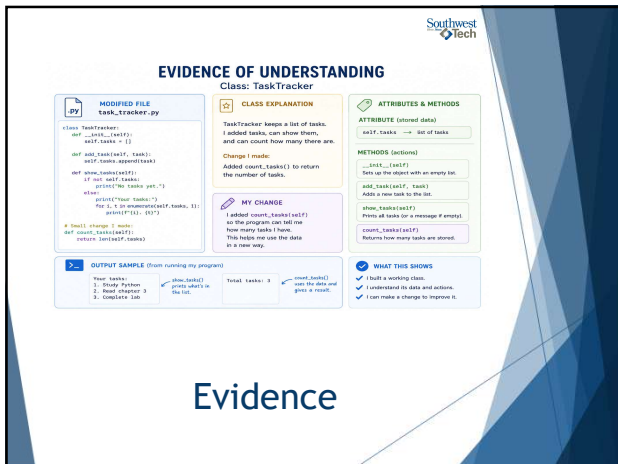
Assignment #7 Evidence

Useful evidence includes:

- ▶ modified `.py` file
- ▶ short explanation of the class
- ▶ attributes identified
- ▶ methods identified
- ▶ one working modification
- ▶ comparison to procedural or function-based structure

Explain the code in your own words.

37



Southwest Tech

EVIDENCE OF UNDERSTANDING

Class: TaskTracker

MODIFIED FILE
task_tracker.py

```

class TaskTracker:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append(task)

    def show_tasks(self):
        if not self.tasks:
            print("No tasks yet.")
        else:
            print("Your tasks:")
            for i, task in enumerate(self.tasks, 1):
                print(f"{i}. {task}")

    # Small change I made:
    def count_tasks(self):
        return len(self.tasks)

# OUTPUT SAMPLE (from running my program)
Your tasks:
1. Study Python
2. Read chapter 3
3. Complete lab
show_tasks()
prints data in the list
Total tasks: 3
count_tasks()
uses the data and gives a result

```

CLASS EXPLANATION

TaskTracker keeps a list of tasks. I added tasks, can show them, and can count how many there are.

Change I made:
Added `count_tasks()` to return the number of tasks.

ATTRIBUTES & METHODS

ATTRIBUTE (internal data)
self.tasks → list of tasks

METHODS (actions)
__init__(self)
Then set the object with an empty list.
add_task(self, task)
Add a new task to the list.
show_tasks(self)
Prints all tasks (or a message if empty).
count_tasks(self)
Returns how many tasks are stored.

MY CHANGE

I added `count_tasks(self)` so the program can tell me how many tasks I have. This helps me use the data in a new way.

WHAT THIS SHOWS

- ✓ I built a working class.
- ✓ I combined the data and actions.
- ✓ I can make a change to improve it.

38



Southwest Tech

Thank you!

Have Fun!

39
